Write-up: Model Documentation

1. Who are you?

For basic information we have the following:
Our names are: Rein Viegers, Maxim Cardenas Cruz and Willem Dieleman.
Our home town is Delft, Netherlands.
Images are attached to this reply, file name states to whom they belong.
For the socials we have our team's website: https://www.teamepoch.ai/, and our personal linkedin's: https://www.linkedin.com/in/willem-dieleman/, https://www.linkedin.com/in/rein-viegers/ and https://www.linkedin.com/in/maxim-cardenas-562194217/.

We are team Epoch VI, the AI dream team at TU Delft. We are a team students that set aside an entire year during our studies to work on AI/ML competitions and projects. This includes doing these ML competitions, and we have already won some competitions in the past, like the segmenting kelp forest competition. 3 engineers worked on the phonetic track in this competition while other 4 worked on the word track. Our team changes every year, and we are the 6th generation of this team. Our backgrounds are in Aerospace Engineering and Computer Science.

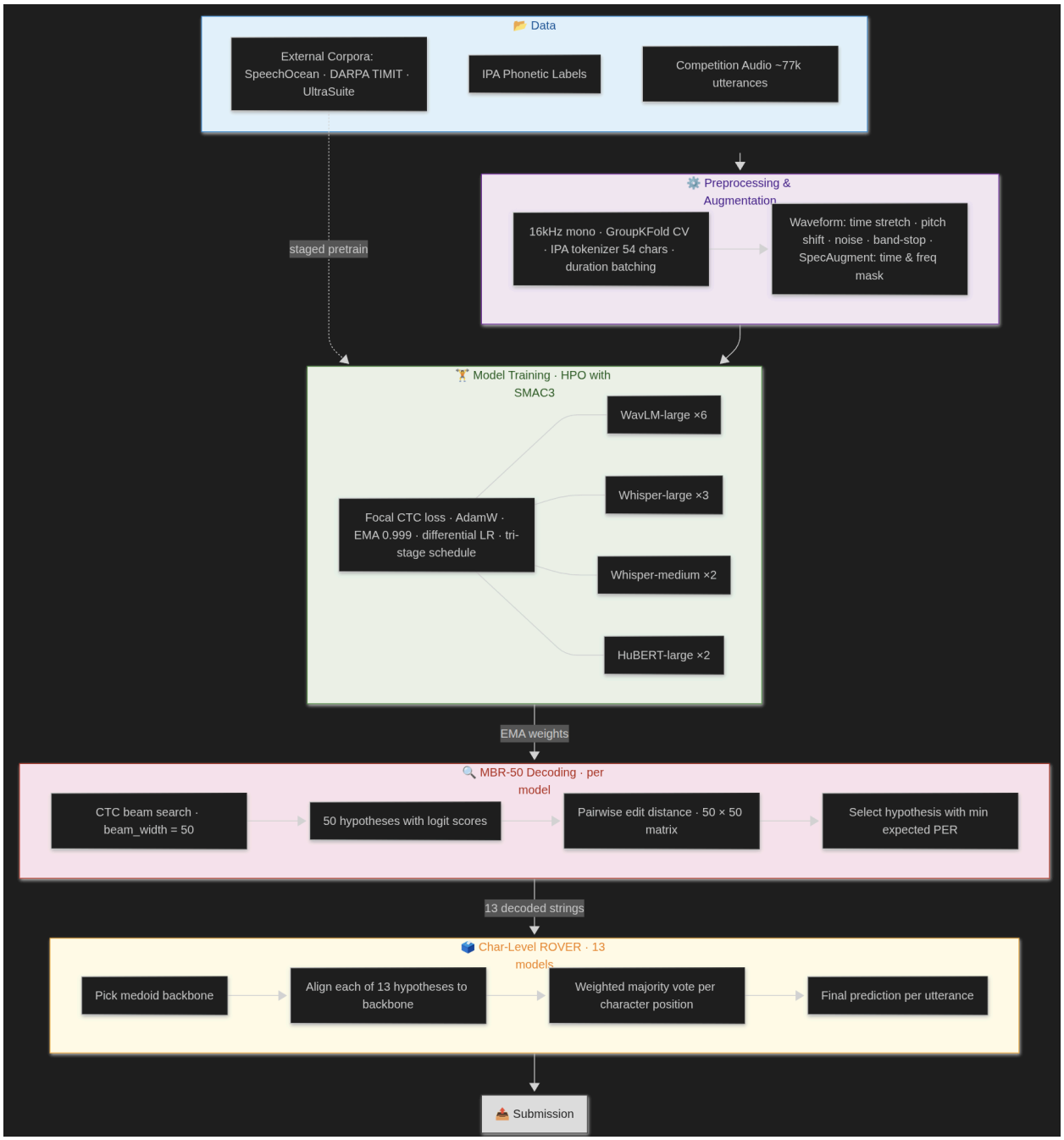
2. Motivation

When we are selecting competitions, we look at various things, like overall societal impact, timeframe and personal interest. This competition fitted perfectly into our schedule and was a completely new challenge for all of us.

3. High-Level Approach

Our basic pipeline consisted out of training multiple CTC-based speech recognition models, decoding the logits using Minimum-Bayes-Risk beam search (50 width) and ensembling the models using ROVER. Our final ensemble consisted out of 13 models, 6 Wavlm-large, 2 Hubert-large, 3 Whisper-Large-v3 and 2 Whisper-medium. All of these models were trained with EMA, waveform augmentations (time stretch, pitch shift, band-stop filter), and a fixed 30sec batch size. Preprocessing only consisted out of converting everything to 16k and Mono.

4. Visualizations



5. Three Most Impactful Code Segments

The way we ensembled our models helped majorly in the final stages, improving our final score by 0.012.

```python
WHISPER_IDX = {6, 7, 8, 9, 10}
n_models = len(output_paths)
logger.info(f"Running char-level ROVER on {len(predictions)} utterances across
{n_models} models...")
logger.info(f"Whisper 1.5x boost for median pred length <= 10")
final_predictions = {}
for utt_id, hyps in predictions.items():
    med_len = sorted(len(h) for h in hyps)[len(hyps) // 2]
    weights = [1.5 if i in WHISPER_IDX and med_len <= 10 else 1.0 for i in
range(n_models)]
    final_predictions[utt_id] = char_rover(hyps, weights)
```

The rover here worked in the following way:

1. **Backbone selection**: pick the medoid hypothesis (minimum total edit distance to all others)
2. **Independent alignment**: align each other hypothesis to the backbone via edit distance
3. **Weighted voting**: at each character position, vote with optional per-model weights
4. **Insertion handling**: include inserted characters only if strict majority votes for them

Second thing that worked very well in our case only using an autocaster with Float16 and a scaler, instead of BFloat16. This improved runs by 0.03 PER when running the same config twice with the different number formats. Why this exactly happens, we dont know.

Apart from that, we just applied the basics very well, so it wasn't that one thing helped us a lot, but many things helped us a little. This included things like EMA, Focal-loss with a low gamma, wave-form augmentation, background noise augmentation, tri-stage learning-rate, different learning rates for head and backbone, gradient clipping and training for 15 epochs.

6. Machine Specs & Time

- CPU (model): Ryzen 9 7950x, Threadripper pro 3945WX, Threadripper pro 5955WX, Threadripper pro 7965WX
- GPU (model or N/A): Quadro RTX 6000 24gb, RTX A5000 24gb, RTX A6000 48gb
- Memory (GB): 96 - 128 GB DDR4/5
- OS: Linux Ubuntu 24.04.4 LTS
- Train duration: Depending on the model and GPU 5-12 hours.
- Inference duration: Slightly less than the 2 hour timelimit.

7. Caveats & Known Issues

If you are training a new model with our pipeline, setting your learning rate to high can cause the model to collapse. All the pretrained models should be stable and run on any GPU that has enough VRAM. Running the preprocessing script beforehand isn't strictly necessary, but will speed up inference and training time significantly. The data will be cached in memory and the training roughly used 20-30 GB of RAM.

8. Tools for Data Preparation / EDA

No. We just used notebooks and python files to explore data and analyze results.

9. Performance Evaluation Beyond Competition Metric

We used the local metric as our guide to decide what works and what doesn't. As for actual error analsysis, we used derivatives of the error metrics, like splitting it per age group, duration and looking at patterns in the errors, like certain substitutions, insertions and deletions.

10. Things Tried That Didn't Make Final

We tried a lot of different things, too many to list in a quick overview, but the main ideas deal with fitting the word-track data into our pipeline. We tried to pretrain using SSL on the word-track data, and using MTL with the word-track data. We scrapped this idea due to time constraints, but it showed potential. The second set of ideas all deal with post-processing, like optimizing decoder parameters, post-processing rules and ensemble methods. Finally, we tried some external datasets, but non of them helped.

11. Future Improvements

We would spend more time on figuring out most sophisticated preprocessing pipeline and really diving deep into the SSL and MTL idea's. We also looked at audio alignment and segmentation, like for example being able to filter out any teachers speaking in the background. This could have also been a feature of the data. Secondly, we need to figure out a solution for the length-mark phoneme, which is currently the phoneme we are struggling with the most.

12. Simplifications for Faster Inference

The amount of models in the ensemble could be reduced, which could save time. An ensemble of only 9 models gave a worse score of only 0.0014. Each model on its own was able to generate predictions for the 77000 test samples in around 9 minutes. A speedup here could also be achieved by preprocessing and converting everything to a .npy file beforehand.

Additionally, we used beam-width 50 for our decoding, and that can be reduced to 10-15 without a major performance degegration.